

AMSS 2026 — Examen restanță (model)

Soluții și barem orientativ

Traian-Florin Șerbănuță

2026

Acestea sunt **soluții model pentru subiectul-model** (domeniul bike-sharing din ‘examen-2026.pdf’). Subiectul real de la restanță folosește **alt scenariu** — dar **baremul și modul de notare sunt aceleași**. Folosește acest document ca să înțelegi *ce se așteaptă* de la tine, nu ca listă de răspunsuri de memorat.

Cum se notează (barem orientativ)

Examenul oglindește rubrica apărării orale (F1 + F3 + F4). Se notează **competența**, nu prezentarea. Pondere orientativă:

Partea I (F1) — ~50%

Identificarea defectelor: cel puțin cinci defecte reale, fiecare cu *categorie*, *severitate* și *corecție*. Răspuns slab: „arată ciudat” fără categorie sau corecție.

Partea II (F3) — ~25%

O *regulă de domeniu concretă* plus un *criteriu clar de respingere*. Răspuns slab: „aș cere AI-ului o diagramă corectă”, fără regulă și fără criteriu.

Partea III (F4) — ~25%

Identificarea inconsistenței *plus* parcurgerea lanțului de trasabilitate *plus* corecția. Răspuns slab: „multiplicitatea e greșită”, fără traseu.

Partea I — Critică (F1): soluție model

Defectele plantate în artefactele-model (minim cinci așteptate):

Diagrama de clase.

Defect (categorie)	Severitate	Corecție
clasă-zeu: <code>BikeSystem</code> (ține tot + <code>doEverything()</code>)	critică	distribuie responsabilitățile pe <code>Rental</code> , <code>Bike</code> , <code>Station</code> ; elimină clasa-zeu
multiplicitate greșită: <code>Rental "*" -- "*" Bike</code>	critică	R1: o închiriere = o bicicletă → <code>Rental "*" -- "1" Bike</code>
agregare lipsă: <code>Station -- Bike</code> (linie simplă)	majoră	R2: stația <i>deține</i> biciclete care îi supraviețuiesc → <code>Station o-- Bike</code>
asociere vagă: <code>Rental</code> → <code>Payment</code> fără multiplicitate	minoră	precizează multiplicitatea (o închiriere are o plată)

Diagrama de stare (Bike).

Defect (categorie)	Severitate	Corecție
stare orfană / cerință nemodelată: nimic nu intră în <code>Maintenance</code> (R3 cere ca o bicicletă cu defect să intre în mentenanță)	majoră	adaugă <code>InUse --> Maintenance : fault</code>
lipsă pseudo-stare inițială: nu există <code>[*]</code> → <code>Available</code>	majoră	adaugă starea inițială

Partea II — Raționament (F3): răspuns model

R3: o bicicletă cu defect intră în mentenanță și revine disponibilă după reparație.

Aș specifica regula de domeniu explicit: „din `InUse`, o bicicletă cu defect trece în `Maintenance`; din `Maintenance` revine în `Available` după reparație; mașina are o stare inițială (`Available`).” Aș cere ca fiecare stare să fie *accesibilă* și să aibă o ieșire.

Aș **respinge**: o stare `Maintenance` fără tranziție de intrare (orfană); absența stării inițiale; tranziții fără eveniment; stări inventate care nu derivă din R3.

Se punctează: regula de domeniu concretă și criteriul de respingere. „Aș cere o diagramă bună” nu primește punctaj.

Partea III — Trasabilitate (F4): răspuns model

R1: o închiriere este pentru exact o bicicletă.

R1 spune *exact o bicicletă*. Diagrama arată **Rental** "*" -- "*" **Bike** (mulț-la-mulți): o închiriere ar putea referi mai multe biciclete. **Inconsistență directă.**

Lanțul de trasabilitate: R1 (cerință) → use case „Închiriază bicicletă” → clasa **Rental** și relația ei cu **Bike**. Citind multiplicitatea cu voce tare — „o închiriere are mai multe biciclete?” — contrazice R1. Corecție: **Rental** "*" -- "1" **Bike**.

Se punctează: identificarea inconsistenței *plus* parcurgerea lanțului cerință→model *plus* corecția.